JSONB in PostgreSQL: guida pratica

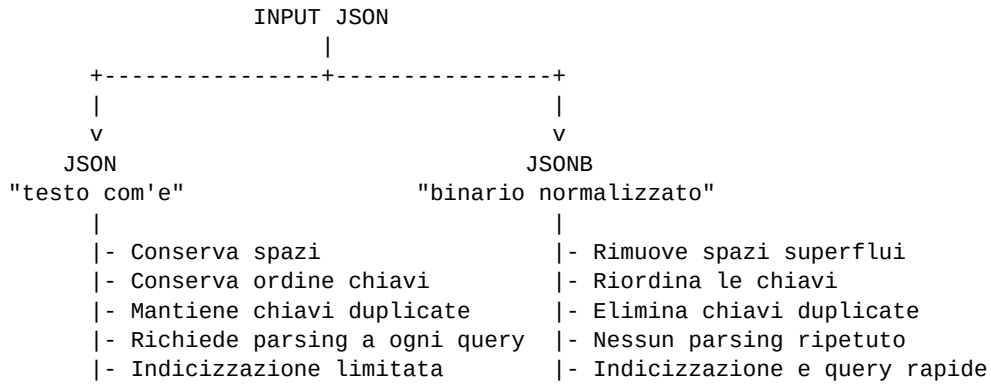
Questa guida raccoglie le differenze tra JSON e JSONB, le tecniche di accesso ai dati JSON, gli operatori di confronto e il funzionamento interno degli indici GIN. Per la maggior parte dei casi applicativi, JSONB è la scelta preferita perché supporta query, aggiornamenti e indici efficienti.

1. JSON e JSONB

In PostgreSQL, json memorizza il testo JSON così come è stato inserito, mentre jsonb lo normalizza in una rappresentazione binaria ottimizzata per query, indici e operatori. Per attributi flessibili, filtri, ricerche e payload API interrogabili, jsonb è normalmente la scelta migliore.

Aspetto	json	jsonb
Memorizzazione	Copia testuale esatta dell'input	Rappresentazione binaria normalizzata
Query	Richiede il parsing del testo a ogni lettura	Non richiede riparsing: più veloce
Scrittura	Più rapida	Leggermente più lenta per la conversione
Indici	Limitati	Supporta GIN e operatori JSONB
Spazi e formattazione	Conservati	Eliminati
Ordine delle chiavi	Conservato	Non garantito
Chiavi duplicate	Conservate	Mantiene solo l'ultimo valore
Uso tipico	Archivio o output fedele all'input	Dati semistrutturati interrogabili

Riepilogo grafico



Esempio

```
SELECT
  '{"b": 2, "a": 1, "a": 3, " nota ": "x"}'::json AS json_colonna,
  '{"b": 2, "a": 1, "a": 3, " nota ": "x"}'::jsonb AS jsonb_colonna;
```

Con json, l'ordine, gli spazi e le chiavi duplicate restano visibili. Con jsonb, PostgreSQL normalizza il documento e mantiene solo l'ultimo valore della chiave duplicata.

2. JSON path e subscripting

JSON path e subscripting sono due modi per raggiungere valori dentro una colonna jsonb. Il primo usa operatori come ->, ->>, #> e #>>; il secondo usa la sintassi ad array colonna['chiave'][0].

Schema di esempio

```
CREATE TABLE ordini (
  id int PRIMARY KEY,
  dati jsonb
);

INSERT INTO ordini (id, dati) VALUES
(1, '{
  "cliente": {"nome": "Anna"},
  "articoli": [
    {"sku": "A1", "prezzo": 10},
    {"sku": "B2", "prezzo": 25}
  ]
}');
```

Subscripting

```
-- Oggetto annidato
SELECT dati['cliente']['nome']
FROM ordini WHERE id = 1;
-- "Anna"

-- Elemento di un array: indice da 0
SELECT dati['articoli'][0]['sku']
FROM ordini WHERE id = 1;
-- "A1"

-- Ultimo elemento: indice negativo
SELECT dati['articoli'][-1]['prezzo']
FROM ordini WHERE id = 1;
-- 25
```

Gli indici partono da 0; gli interi negativi contano dalla fine dell'array. Non sono supportate le slice tipo [1:3]. Il risultato dell'accesso tramite [] su jsonb è sempre jsonb.

Modifica con subscripting

```
UPDATE ordini
SET dati['articoli'][0]['prezzo'] = '12'
WHERE id = 1;
```

Il percorso deve essere già "attraversabile": se una chiave o un indice intermedio non esiste, l'aggiornamento non crea automaticamente tutta la struttura.

Operatori di percorso

Gli operatori -> e #> restituiscono jsonb; ->> e #>> restituiscono text. Usa -> e #> per continuare a navigare o conservare il tipo JSON; usa ->> e #>> per ottenere un valore SQL normale.

Operatore	Restituisce	Uso
->	jsonb	Estrae un elemento o oggetto JSON
->>	text	Estrae il valore come testo
#>	jsonb	Estrae un valore a un percorso annidato
#>>	text	Estrae un valore annidato come testo

```
-- jsonb: "Anna"
```

```
SELECT dati -> 'cliente' -> 'nome'
FROM ordini WHERE id = 1;

-- testo: Anna
SELECT dati -> 'cliente' ->> 'nome'
FROM ordini WHERE id = 1;

-- jsonb: {"sku": "A1", "prezzo": 10}
SELECT dati -> 'articoli' -> 0
FROM ordini WHERE id = 1;

-- testo: A1
SELECT dati #>> '{articoli,0,sku}'
FROM ordini WHERE id = 1;

-- jsonb: {"sku": "A1", "prezzo": 10}
SELECT dati #> '{articoli,0}'
FROM ordini WHERE id = 1;
```

SQL/JSON Path

```
SELECT jsonb_path_query(
  dati,
  '$.articoli[*] ? (@.prezzo > 20)'
)
FROM ordini
WHERE id = 1;

-- Risultato:
-- [{"sku": "B2", "prezzo": 25}]
```

In $ si indica la radice del documento; $.articoli[*] seleziona tutti gli elementi dell'array; ? (@.prezzo > 20) è un filtro. Il tipo jsonpath è utile per query più espressive e condizioni complesse.

Scelta rapida

```
Vuoi leggere/modificare un percorso noto?
|- Percorso semplice e leggibile ----------------> dati['cliente']['nome']
|- Estrazione come jsonb -----------------------> dati -> 'cliente' -> 'nome'
|- Estrazione come testo -----------------------> dati -> 'cliente' ->> 'nome'
|- Percorso annidato compatto ------------------> dati #>> '{articoli,0,sku}'
|- Filtro, wildcard o logica avanzata ----------> jsonb_path_query(..., '$.articoli[*] ? (...)')
```

3. Operatori di confronto JSONB

Gli operatori di confronto per jsonb comprendono gli operatori SQL standard, gli operatori di contenimento, gli operatori di esistenza e gli operatori JSON path. Non sono disponibili per il tipo json.

Confronto ordinale

```
SELECT *
FROM eventi
WHERE payload = '{"stato": "attivo"}'::jsonb;
```

Gli operatori =, <>, <, <=, > e >= confrontano due valori jsonb secondo le regole di ordinamento B-tree di PostgreSQL. L'ordinamento segue una gerarchia di tipi: oggetto, array, booleano, numero, stringa e null.

Contenimento

```
-- Il documento contiene il pattern
SELECT *
FROM utenti
WHERE profilo @> '{"ruolo": "admin"}'::jsonb;

-- Il documento è contenuto nel pattern
SELECT *
FROM utenti
WHERE profilo <@ '{"ruolo": "admin", "attivo": true}'::jsonb;
```

L'operatore @> verifica che il valore a sinistra contenga quello a destra; <@ verifica il contrario. Per gli array, l'ordine non conta e i duplicati vengono trattati una sola volta.

Esistenza di chiavi

```
-- Ha il campo "email"
SELECT * FROM utenti WHERE profilo ? 'email';

-- Ha almeno uno tra "email" e "telefono"
SELECT * FROM utenti
WHERE profilo ?| ARRAY['email', 'telefono'];

-- Ha sia "nome" sia "cognome"
SELECT * FROM utenti
WHERE profilo ?& ARRAY['nome', 'cognome'];

-- L'array contiene la stringa "admin"
SELECT * FROM utenti
WHERE ruoli ? 'admin';
```

Questi operatori cercano chiavi di primo livello negli oggetti oppure elementi di primo livello negli array. Non effettuano ricerche ricorsive nei sotto-oggetti.

JSON path

```
-- Esiste almeno un articolo con prezzo > 20?
SELECT *
FROM ordini
WHERE dati @? '$.articoli[*] ? (@.prezzo > 20)';

-- Tutti gli articoli hanno prezzo > 20?
SELECT *
FROM ordini
WHERE dati @@ 'exists($.articoli[*] ? (@.prezzo <= 20)) = false';
```

L'operatore @? verifica l'esistenza di almeno un risultato del JSON path; @@ valuta un predicato booleano e restituisce true solo se il risultato è vero.

Riepilogo grafico

```
CONFRONTO JSONB
|
|-- Ordine / uguaglianza
|   |-- =      uguale
|   |-- <>     diverso
|   |-- <      minore
|   |-- <=     minore o uguale
|   |-- >      maggiore
|   |-- >=     maggiore o uguale
|
|-- Contenimento
|   |-- @>     A contiene B
|   |-- <@     A e' contenuto in B
|
|-- Esistenza a primo livello
|   |-- ?      una chiave/elemento
|   |-- ?|     almeno una tra piu' chiavi/elementi
|   |-- ?&     tutte le chiavi/elementi
|
|-- JSON path
    |-- @?     esiste almeno un risultato del path
    |-- @@     il predicato path e' vero
```

4. Internals degli indici GIN

Un indice GIN (Generalized Inverted Index) è un indice invertito: invece di mappare una riga ai suoi valori, mappa ogni chiave estratta dal valore alle righe che la contengono. È particolarmente adatto a array, full-text search e JSONB.

Struttura

```
Tabella heap
--------------------------------------------
TID 1: {"tags": ["sql", "postgres"]}
TID 2: {"tags": ["postgres", "jsonb"]}
TID 3: {"tags": ["jsonb", "performance"]}

Indice GIN
--------------------------------------------
Chiave "sql"         -> [TID 1]
Chiave "postgres"    -> [TID 1, TID 2]
Chiave "jsonb"       -> [TID 2, TID 3]
Chiave "performance" -> [TID 3]
```

Il GIN è costruito come un B-tree sulle chiavi. Per ogni chiave, la foglia contiene una posting list oppure un riferimento a un posting tree.

Componente	Ruolo	Quando si usa
Chiave	Valore estratto dall'oggetto indicizzato	Sempre
Posting list	Lista compatta di TID salvata accanto alla chiave	Pochi riferimenti
Posting tree	B-tree separato di TID per una stessa chiave	Molti riferimenti
B-tree principale	Indice ordinato delle chiavi	Sempre

Lo stesso TID può comparire in più posting list, perché una riga può contenere più chiavi. Per chiavi molto frequenti, la posting list viene sostituita da un posting tree per mantenere efficiente l'accesso.

Fast update e pending list

```
INSERT "nuova riga"
        |
        v
Pending list (scritture rapide)
        |
        v quando viene svuotata
B-tree principale + posting list/tree (query piu' efficienti)
```

Le nuove voci possono essere accumulate in una pending list non ordinata per rendere le scritture più rapide. La lista viene svuotata da VACUUM, ANALYZE, gin_clean_pending_list() oppure quando supera gin_pending_list_limit.

```
SELECT gin_pending_list_limit;
SELECT * FROM pg_stat_gin_index('idx_utenti_profilo');

-- Forza il consolidamento della pending list
SELECT gin_clean_pending_list('idx_utenti_profilo');
```

Impatto pratico

```
Molte scritture con fastupdate attivo
-> inserimenti piu' rapidi, ma pending list che cresce

Pending list molto grande
-> le query devono esaminarla oltre al B-tree principale
```

VACUUM, ANALYZE o svuotamento manuale
-> le voci vengono consolidate nella struttura principale

fastupdate = off
-> le nuove voci non finiscono nella pending list,
   ma quelle esistenti non vengono svuotate automaticamente

5. jsonb_ops e jsonb_path_ops

jsonb_ops e jsonb_path_ops sono due classi di operatori GIN per colonne jsonb. Indicizzano il documento in modi diversi e supportano query diverse: jsonb_ops è più flessibile, jsonb_path_ops è più compatto e spesso più efficiente per contenimento e JSON path.

Aspetto	jsonb_ops	jsonb_path_ops
Cosa indicizza	Ogni chiave e ogni valore	Ogni valore e la struttura/path
Dimensione indice	Generalmente maggiore	Generalmente minore
Operatori	@>, ?, ?|, ?&, @?, @@	@>, @?, @@
Ricerca per chiave	Sì	No
Prestazioni su @>	Buone	Spesso migliori
Predefinita	Sì	No

Esempio

```
CREATE INDEX idx_profilo_ops
ON utenti USING gin (profilo jsonb_ops);

CREATE INDEX idx_profilo_path_ops
ON utenti USING gin (profilo jsonb_path_ops);

{
  "ruolo": "admin",
  "tag": ["sql", "postgres"]
}
```

Con jsonb_ops vengono create voci indipendenti per la chiave ruolo, il valore admin, la chiave tag e i valori sql e postgres. Con jsonb_path_ops vengono create voci basate sui valori e sulla loro struttura, non una voce autonoma per la sola chiave ruolo.

Query supportate

```
-- Supportato da entrambe le classi
SELECT * FROM utenti
WHERE profilo @> '{"ruolo": "admin"}'::jsonb;

-- Supportato solo da jsonb_ops
SELECT * FROM utenti
WHERE profilo ? 'ruolo';

-- Supportato solo da jsonb_ops
SELECT * FROM utenti
WHERE profilo ?| ARRAY['ruolo', 'tag'];

-- Supportato da entrambe le classi
SELECT * FROM utenti
WHERE profilo @? '$.tag[*] ? (@ == "postgres")';
```

PostgreSQL può usare un indice GIN solo se l'operatore della query è supportato dalla classe dell'indice. Una query con ? non può quindi usare un indice creato con jsonb_path_ops.

Regola pratica

```
Hai bisogno di "profilo ? 'chiave'"?
|- Si -> jsonb_ops
|- No
   |- Query prevalenti: @>, @?, @@
   |  |- jsonb_path_ops: indice piu' piccolo e spesso piu' veloce
   |- Query miste o non definite
```

|- jsonb_ops: scelta piu' flessibile